



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI Agents Development & Agentic Automation

Eng. Fatma Elraey

Lecture 2 — Stateful Agents & Frameworks (LangGraph / CrewAI)

Field	Detail
Instructor	Fatma Elraey
Course	AI Agents Development (3-Day Course)
Session	Day 2 of 3 — Lecture, 3 Hours
Focus	Moving beyond single-turn agent loops into explicit state, graphs, short-term memory, thread persistence, and human-in-the-loop checkpoints.

Quick Recap — Day 1

Day 1 built the core agent loop manually: the model selects a tool, Python executes it, the tool result becomes an observation, and the cycle continues. Today we make that process easier to control across many steps by representing the workflow as state and graph transitions.

Today's Outline

- 1. Why Stateful Agent Architecture Matters
- 2. LangGraph: State, Nodes, Edges & Checkpointers
- 3. Short-Term Memory & Thread Persistence
- 4. Human-in-the-Loop Interrupts
- 5. Where CrewAI Fits

01 Why Stateful Agent Architecture Matters

Previously:

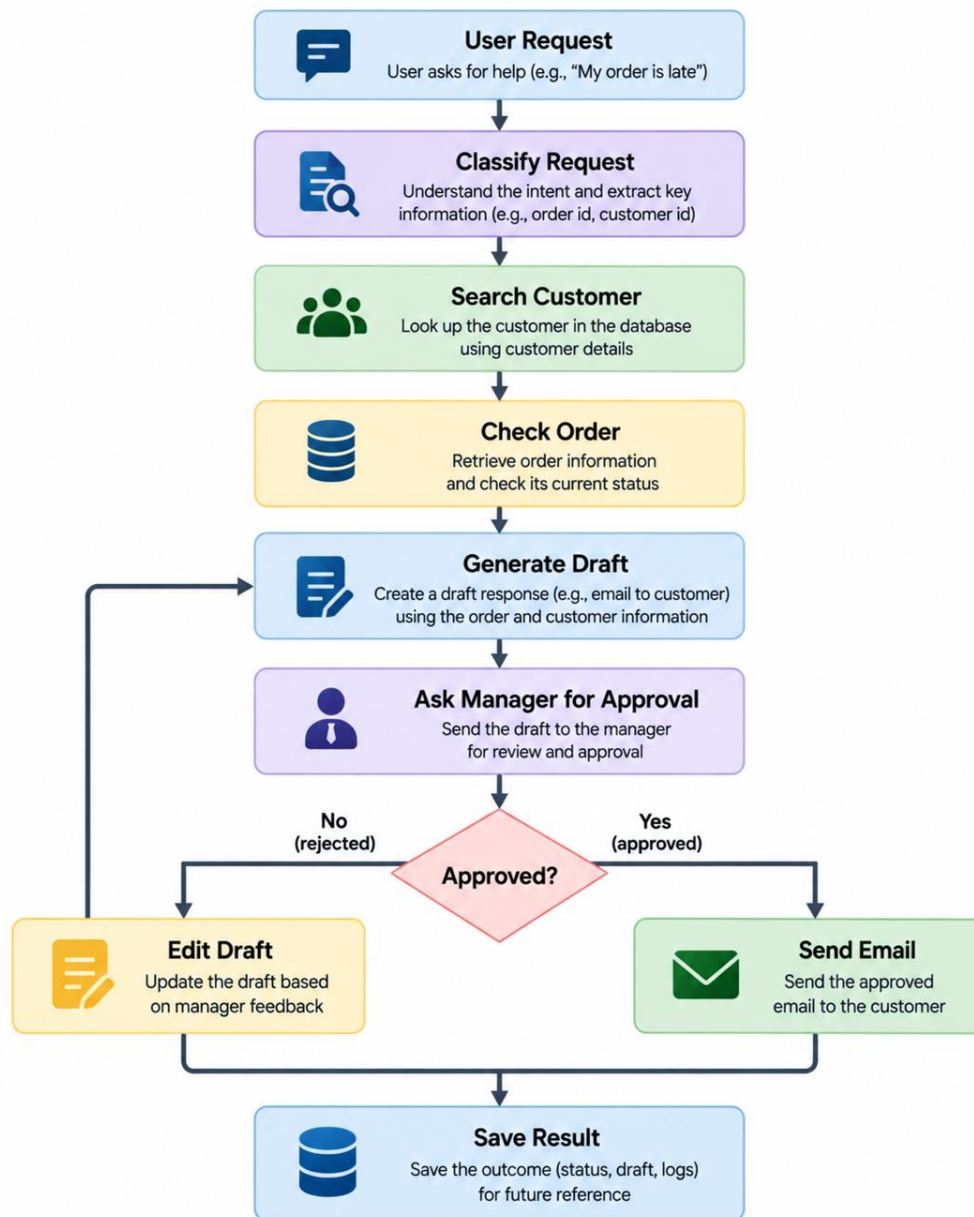


```

while True:
    response = ask_model()

    if response_has_tool_call:
        result = execute_tool()
        add_result_to_context()
    else:
        break
  
```

but let's imagine the workflow like this:



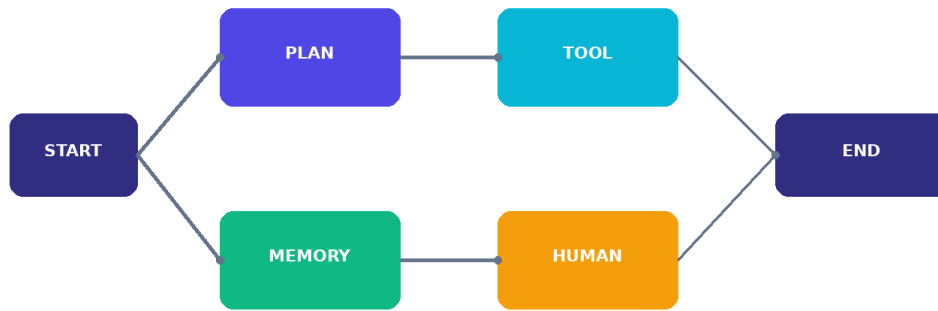
If we did using python , it may look like this:

```
while ...
if ...
elif ...
if ...
try ...
except ...
```

the code becomes very hard to maintain. And this is why we are getting into **Stateful Agent Architecture**.

A manual while-loop works well for learning and small agents. As workflows grow, you need to know which step is running, what data has been produced, where execution should continue after an error, and how to pause safely. Stateful frameworks make those concerns explicit.

Stateful Agent Graph



Nodes update shared state; edges decide what runs next; a checkpointer persists state by thread.

Figure 2.1 — A stateful agent is a graph whose nodes read and update shared state.

- **State** — the structured data carried through the workflow: messages, customer ID, draft text, approval status, tool results, and more.

Ex:

```

state = {
  "customer_id": 101,
  "question": "Where is my order?",
  "order_status": "delayed",
  "draft_email": "...",
  "approved": False
}
  
```

- **Node** — one unit of work, such as call the model, search a database, draft an email, or ask a human.

- call_model
- search_database
- draft_email
- ask_human
- send_email

- **Edge** — the transition that decides which node runs next.

- START → draft → END
- approval
 - — approved → send
 - — rejected → cancel

- **Checkpointer** — persists state so a thread can continue later instead of restarting from zero.

Draft created



Waiting for manager approval

So if the manages may respond after 5 minutes or after an hour. We don't want the program to be stuck. So save the state and then pause. After the manager reply, we load the state and resume.

Instructor example — why a graph helps

MANUAL LOOP Easy at first, but branching, retries, and approval logic become nested if/else blocks.

GRAPH “classify → lookup → draft → approval → send” is visible as named nodes and explicit transitions.

PRO TIP Use a framework when workflow state and control flow become more complex than the model call itself. Do not add a framework just because the word “agent” appears in the project.

✓ QUICK CHECK

Q1. What does state represent?

Answer: The structured information the workflow needs to carry between steps.

Q2. What is an edge?

Answer: A transition that determines which node runs next.

02 LangGraph: State, Nodes, Edges & Checkpointers

LangGraph models an agent workflow as a state machine. You define a state type, add node functions that return updates, connect those nodes with edges, and compile the graph. A checkpointer can persist each thread so execution can resume later.

First :

```
pip install langgraph
```

```
# Minimal stateful LangGraph
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class SupportState(TypedDict):
    question: str
    draft: str
```

```
def draft_answer(state: SupportState):
    return {"draft": f"Draft for: {state['question']}"}

builder = StateGraph(SupportState)
builder.add_node("draft", draft_answer)
builder.add_edge(START, "draft")
builder.add_edge("draft", END)

# START → draft → END

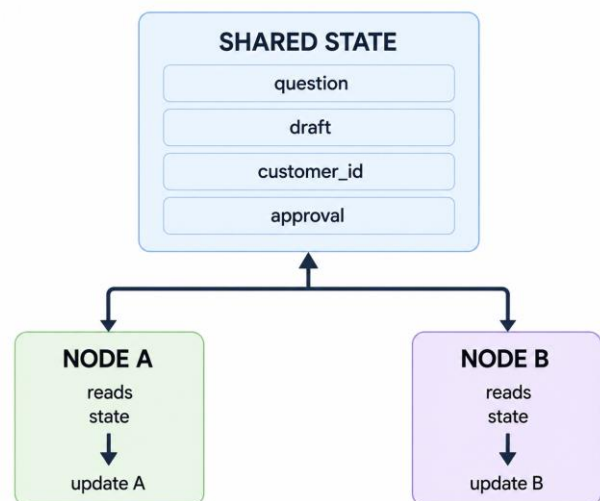
graph = builder.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "support-42"}}
result = graph.invoke({"question": "Where is my order?", "draft": ""}, config)
print(result["draft"])
```

Output:

```
Draft for: Where is my order?
```

The node receives current state and returns only the keys it wants to update. The graph runtime applies those updates and follows the configured edge. Keeping node inputs and outputs explicit makes execution easier to test and debug.



Instructor example — graph vocabulary

STATE The shared notebook.

NODE A worker that reads the notebook and writes an update.

EDGE The route to the next worker.

CHECKPOINTER The saved snapshot that lets the same thread continue later.

✓ QUICK CHECK

Q1. What does a node normally return?

Answer: A state update, not an entirely new application.

Q2. Why compile with a checkpointer?

Answer: To persist thread state across invocations and support pause/resume workflows.

03

Short-Term Memory & Thread Persistence

Memory in a stateful agent is usually not one magical feature. Short-term memory means carrying the information needed for the current conversation or task. Persistence means saving that state under a thread identifier so later invocations can continue the same logical run.

- **Thread ID** — a stable identifier that tells the checkpointer which conversation/run the new call belongs to.
- **Short-term memory** — recent messages, decisions, and intermediate results needed for the current thread.

User: My order number **is** 104.

Agent: Okay.

User: What **is** its status?

And the models knows what is its is for.

- **Persistent checkpointer** — a database-backed saver used in production so state survives process restarts.
- **State trimming** — long threads may need summarization or selective retention to control context size and cost.

Thread identity separates conversations

```
config_a = {"configurable": {"thread_id": "customer-101"}}
```

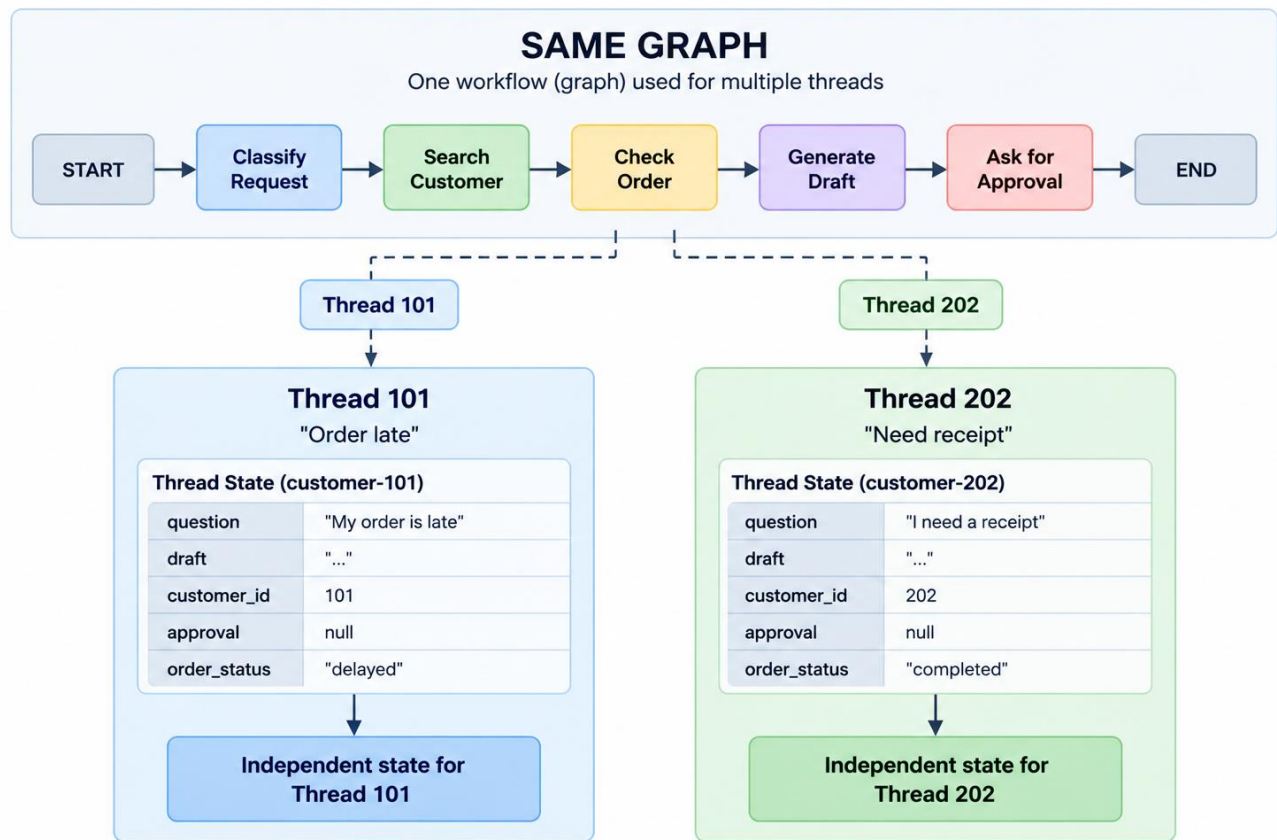
```
config_b = {"configurable": {"thread_id": "customer-202"}}
```

Same graph, different persistent threads.

```
graph.invoke({"question": "My order is late", "draft": ""}, config_a)
```

```
graph.invoke({"question": "I need a receipt", "draft": ""}, config_b)
```

Reuse config_a to continue customer-101 later.



PRO TIP Do not confuse model context with application memory. The framework can persist state, but you still decide what should be saved, what is safe to store, and what the model should see on the next step.

✓ QUICK CHECK

Q1. Why is thread_id important?

Answer: It associates each invocation with the correct persisted state.

Q2. Does persistence mean every piece of state should be sent to the model?

Answer: No. Persisted application state and model-visible context are separate design decisions.

04 Human-in-the-Loop (HITL) Interrupts

Autonomous does not mean uncontrolled. Sensitive tools should pause before execution and request human approval. LangGraph interrupts persist the current state, return an approval payload to the caller, and later resume from the same thread when a human provides a decision.

Human-in-the-Loop Checkpoint

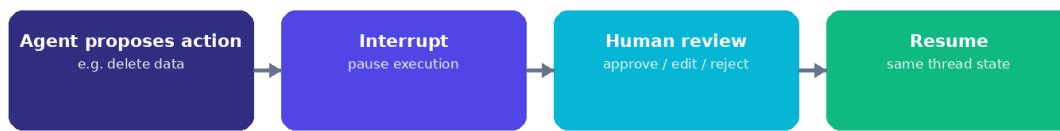


Figure 2.2 — Human approval is a stateful pause, not a new conversation.

```

from typing import Literal, TypedDict
from pprint import pprint

from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt
from langgraph.checkpoint.memory import InMemorySaver

class ApprovalState(TypedDict):
    action: str
    status: str

def approval(
    state: ApprovalState
) -> Command[Literal["proceed", "cancel"]]:

    decision = interrupt({
        "question": "Approve this sensitive action?",
        "action": state["action"],
    })

    return Command(
        goto="proceed" if decision else "cancel"
    )

def proceed(state: ApprovalState):
    print(">>> Proceed node is running")
    return {"status": "approved"}

def cancel(state: ApprovalState):
    print(">>> Cancel node is running")
    return {"status": "rejected"}

builder = StateGraph(ApprovalState)

```

```
builder.add_node("approval", approval)
builder.add_node("proceed", proceed)
builder.add_node("cancel", cancel)

builder.add_edge(START, "approval")
builder.add_edge("proceed", END)
builder.add_edge("cancel", END)

graph = builder.compile(
    checkpointer=InMemorySaver()
)

config = {
    "configurable": {
        "thread_id": "approval-1"
    }
}

# -----
# First run → PAUSE
# -----

paused = graph.invoke(
    {
        "action": "Send customer email",
        "status": "pending"
    },
    config
)

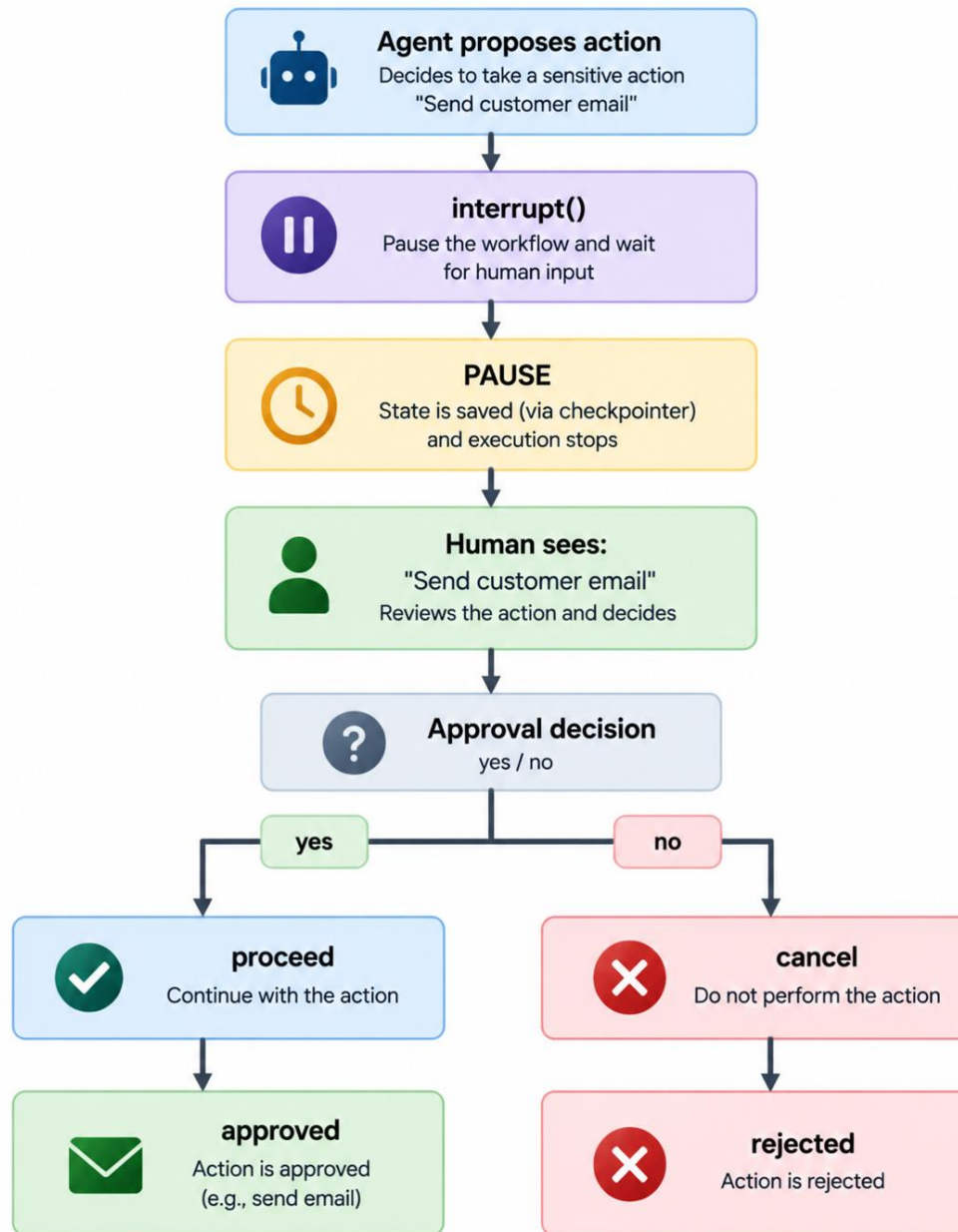
pprint(paused)

answer = input(
    "\nApprove action? (yes/no): "
)

decision = answer.lower() == "yes"

resumed = graph.invoke(
    Command(resume=decision),
    config
)

pprint(resumed)
```



- **Read-only tools** — often safe to run automatically.
- **External side effects** — email, purchases, writes, deletion, or permission changes deserve stronger controls.
- **Approval payload** — show the human exactly what will happen, with enough context to decide.
- **Resume** — use the same thread ID so the workflow continues from the correct checkpoint.

✓ QUICK CHECK

Q1. What happens when `interrupt()` is reached?

Answer: Graph execution pauses and persisted state allows it to resume later.

Q2. What must stay the same when resuming?

Answer: The thread identifier / configuration associated with the paused run.

05

Where CrewAI Fits

LangGraph emphasizes explicit state and graph control. CrewAI provides higher-level concepts for collaborative agent teams: Agents with roles and goals, Tasks that describe assignments, a Crew that coordinates agents, and a Process that defines how tasks are executed. Both can build agentic systems, but their abstraction level is different.

Instructor example — framework choice

LANGGRAPH Best when you want explicit nodes, state transitions, persistence, branching, and approval checkpoints.

CREWAI Useful when the problem naturally looks like a team of specialized agents completing tasks together.

NATIVE PYTHON Still excellent for small, transparent tool loops with minimal dependencies.

CrewAI mental model

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(
    role="Research Analyst",
    goal="Collect the facts needed for a concise briefing",
    backstory="You verify evidence before summarizing it.",
    verbose=True,
)

task = Task(
    description="Prepare a short briefing on the supplied topic.",
    expected_output="Five factual bullets and a short conclusion.",
    agent=researcher,
)

crew = Crew(
    agents=[researcher],
    tasks=[task],
    process=Process.sequential,
)

result = crew.kickoff()
```

PRO TIP Choose the framework that makes control flow clearer. Frameworks are implementation tools, not a measure of how “agentic” a system is.

✓ QUICK CHECK

Q1. What is the main architectural strength of LangGraph?

Answer: Explicit, stateful graph control and persistence.

Q2. What mental model does CrewAI emphasize?

Answer: Specialized agents collaborating on tasks inside a crew/process.

06 Check Your Understanding

Work through these on your own before checking the marked answers.

Practice Question 1

What is the main purpose of a checkpoint?

- A. Train Gemini
- B. Persist graph state for a thread
- C. Replace tools
- D. Create Pydantic schemas

Practice Question 2

Which item is a LangGraph node?

- A. A unit of work that reads/updates state
- B. A secret key
- C. A JSON field only
- D. A browser tab

Practice Question 3

Why use a thread ID?

- A. To choose a Python version
- B. To identify which persisted conversation/run to continue
- C. To disable memory
- D. To rename a tool

Practice Question 4

When should HITL be considered?

- A. Only for read-only search
- B. Before sensitive or irreversible actions
- C. Never in agent systems
- D. Only after deployment

Practice Question 5

Which framework abstraction emphasizes a team of role-based agents and tasks?

- A. Pydantic
- B. CrewAI
- C. SQLite
- D. dotenv

Answers

Practice Question 1: B ✓ — A checkpointer stores state so a workflow can continue or resume.

Practice Question 2: A ✓ — Nodes implement workflow steps and return state updates.

Practice Question 3: B ✓ — The checkpointer uses the thread identifier to associate state with the correct run.

Practice Question 4: B ✓ — Human approval reduces risk for side effects such as send/delete/write actions.

Practice Question 5: B ✓ — CrewAI organizes Agents, Tasks, Crews, and Processes.